Day 3 — Object-Oriented Programming (OOP)

Answers, Explanations, and Java Programs

Classes and objects: A class is a blueprint; an object is an instance of a class.
Constructor: Initializes an object and has the same name as its class, with no return type.
Encapsulation: Keeps data and methods together and controls access, commonly using private fields and public methods.
Inheritance: A child class reuses or extends a parent class with `extends`.
Polymorphism: One interface or method name can have different forms. Overloading is compile-time; overriding supports runtime polymorphism.
Abstraction: Shows essential behavior while hiding implementation details; use abstract classes or interfaces.
Access modifiers: `public` broadly accessible; `protected` same package and subclasses under Java rules; package-private (no modifier) same package; `private` same class only.
Method overloading: Same method name with different parameter lists; changing only the return type is not enough.

Program 1 — Student and Employee classes

Save as `Main.java`. Demonstrates classes, objects, properties, methods, constructors, encapsulation, inheritance, overriding, and overloading.

```
class Student {
    private String name;
    private int age;

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
    public String getName() { return name; }
    public int getAge() { return age; }

    public void displayDetails() {
        System.out.println("Student: " + name + ", Age: " + age);
    }
    public void displayDetails(String course) {
        System.out.println("Student: " + name + ", Course: " + course);
    }
}

class Employee {
    private String name;
    private double salary;

    public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }
    public void displayDetails() {
        System.out.println("Employee: " + name + ", Salary: " + salary);
    }
}

class GraduateStudent extends Student {
    private String specialization;

    public GraduateStudent(String name, int age, String specialization) {
        super(name, age);
        this.specialization = specialization;
    }
    @Override
    public void displayDetails() {
        System.out.println("Graduate Student: " + getName()
                + ", Age: " + getAge()
                + ", Specialization: " + specialization);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Asha", 21);
        s1.displayDetails();
        s1.displayDetails("Computer Engineering");

        Employee e1 = new Employee("Rahul", 35000);
        e1.displayDetails();

        Student s2 = new GraduateStudent("Neha", 23, "Java");
        s2.displayDetails();
    }
}
```

Expected output

```
Student: Asha, Age: 21
Student: Asha, Course: Computer Engineering
Employee: Rahul, Salary: 35000.0
Graduate Student: Neha, Age: 23, Specialization: Java
```

Program 2 — Console-based Student Management Application

This menu program adds and displays one student record. It uses a constructor, private fields, and a loop.

```
import java.util.Scanner;

class StudentRecord {
    private int rollNo;
    private String name;
    private String course;

    public StudentRecord(int rollNo, String name, String course) {
        this.rollNo = rollNo;
        this.name = name;
        this.course = course;
    }
    public void display() {
        System.out.println("Roll No: " + rollNo);
        System.out.println("Name: " + name);
        System.out.println("Course: " + course);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        StudentRecord student = null;
        int choice;

        do {
            System.out.println("\n1. Add student");
            System.out.println("2. Display student");
            System.out.println("3. Exit");
            System.out.print("Enter choice: ");
            choice = sc.nextInt();
            sc.nextLine();

            switch (choice) {
                case 1:
                    System.out.print("Enter roll number: ");
                    int roll = sc.nextInt();
                    sc.nextLine();
                    System.out.print("Enter name: ");
                    String name = sc.nextLine();
                    System.out.print("Enter course: ");
                    String course = sc.nextLine();
                    student = new StudentRecord(roll, name, course);
                    System.out.println("Student added successfully.");
                    break;
                case 2:
                    if (student == null)
                        System.out.println("No student record found. Add a student first.");
                    else
                        student.display();
                    break;
                case 3:
                    System.out.println("Exiting application.");
                    break;
                default:
                    System.out.println("Invalid choice.");
            }
        } while (choice != 3);
        sc.close();
    }
}
```

How to run

Save and run each program separately because both use a public class named `Main`. Compile with `javac Main.java` and run with `java Main`. In an IDE, create a class named Main and paste one program at a time.

Short speaking answers

Class/object: A class is a blueprint; an object is an instance.
Constructor: Initializes an object when it is created.
Encapsulation: Protects data by restricting direct access.
Inheritance: Lets a child class reuse and extend a parent class.
Polymorphism: Allows the same method name or reference to have different behaviors.
Abstraction: Hides implementation details and exposes essential behavior.
Overloading: Same method name with different parameters.